Inspiring Excellence

CSE370 : Database Systems
Project Report
Project Title : HabitCents

Group No : 01, CSE370 Lab Section : 10, Spring 2026		
ID	**Name**	**Contribution**
24141020	Mohammed Sabbir Ahmed	Frontend, Backend, EER Diagram, Normalised Schema
23201430	Sumaya Afroze Chaha	Frontend, Backend, EER Diagram, Normalised Schema
24101526	Monaz Zerin Monir Bismah	Frontend,Backend, EER diagram, Normalised Schema

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	3
3	ER/EER Diagram	4
4	Schema Diagram	5
5	Normalization	6
6	Frontend Development	7-8
7	Backend Development	9-12
8	Source Code Repository	12
9	Conclusion	13
10	References	13-14

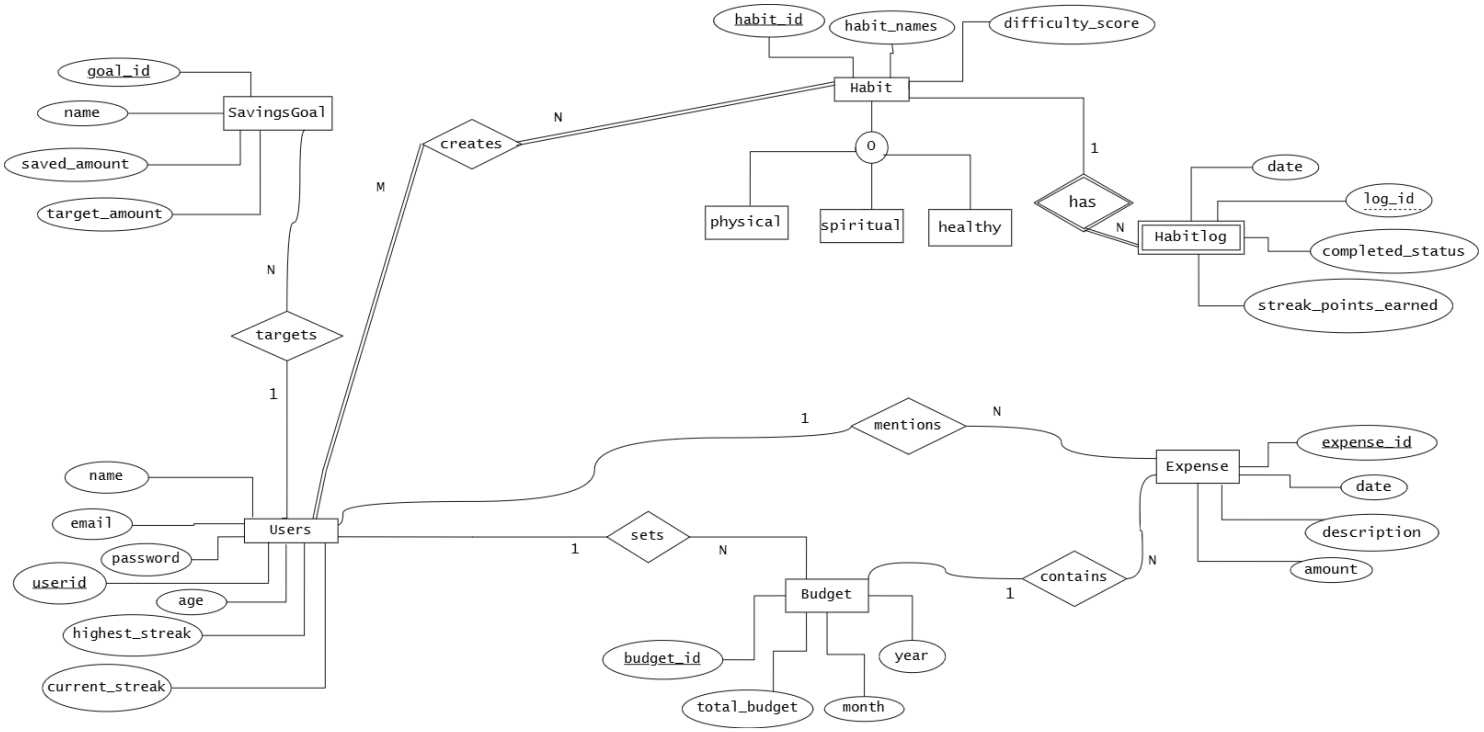
Introduction

HabitCents is a productivity platform that helps users build their positive habits while also contributing in managing their day-to-day financial expenses. The platform allows users to keep track of their daily habits and maintain streaks based on their everyday completion. This platform also helps users kick their bad habits and lead a healthy lifestyle. Along with habit tracking, it also monitors their personal expenses within a fixed monthly budget. Users will be notified through a budget bar and alerted them about overspending. Lastly, it offers a savings feature where users can set a goal amount for their desired purchases.

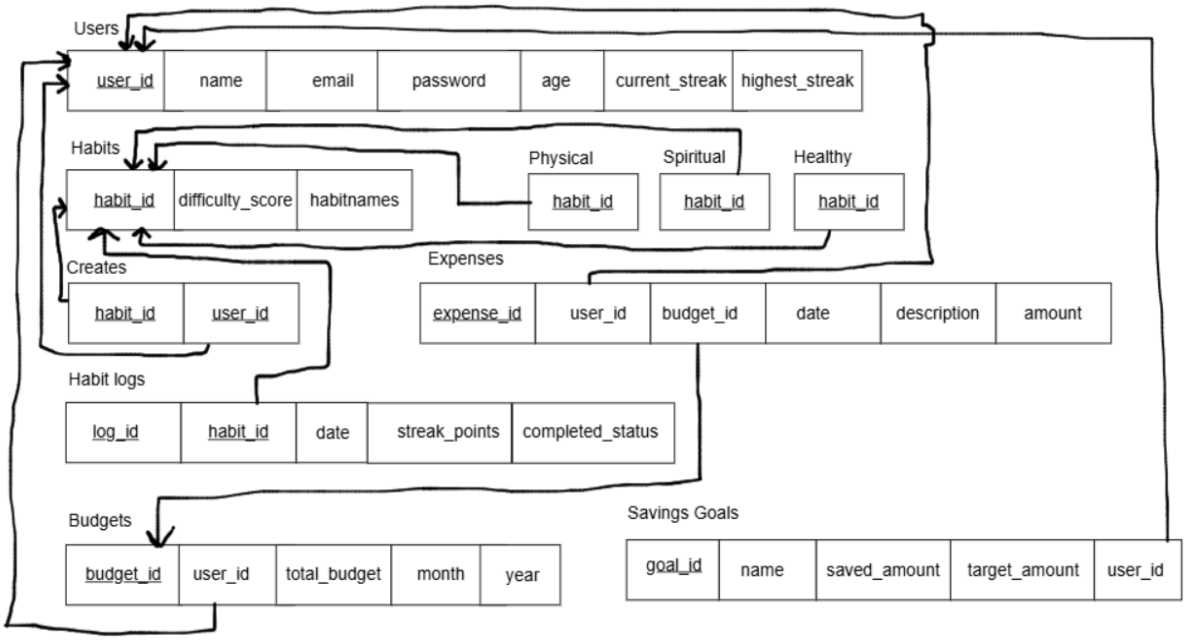
Project Features

ID, Name	Features [3 per member]	
24141020, Mohammed Sabbir Ahmed	Ft 1	Create Habits with Difficulty Scoring
	Ft 2	Mark Habit Completion & Habit Streak Calculation
	Ft 3	Habit Categorization
23201430, Sumaya Afroze Chaha	Ft 1	Expense Tracking
	Ft 2	Set Monthly Budget
	Ft 3	Leaderboard
24101526, Monaz Zerin Monir Bismah	Ft 1	Dashboard
	Ft 2	Over Expenditure Detection
	Ft 3	Savings Calculation

ER/EER Diagram



Schema Diagram



Normalization

a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.

Ans: The converted Schema is in 1NF as we followed the 8 rules while converting the Schema from EER diagram.

b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?

Ans: The schema is in 2NF since all tables are already in 1NF and most have single primary keys. We don't have any partial functional dependencies in our relational schema.

c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

Ans: The schema is in 3NF since all tables are already in 2NF . There are no transitive dependencies in our relational schema.

Frontend Development

Contribution of ID : 24141020, Name : Mohammed Sabbir Ahmed

For frontend development, I went with a blend of html, css and javascript along with some EJS (Embedded JavaScript) for templating. With the help of these technologies along with some external libraries like Bootstrap I built these following webpages in the repository.

- Categories.ejs
- Index.ejs
- Login.ejs
- Signup.ejs
- Habits.ejs

At first, I wrote the whole webpages in html and css and linked the pages. Secondly, after I was happy with the outcome or the way they looked, I noticed there was some redundancy as in similar html code being executed in some of the webpages. For example, the navbar and footer remained the same across the pages. So, I converted them into separate pages (head.ejs, navbar.ejs, footer.ejs) and then linked them back into all the pages using EJS templating. Besides, using EJS also helped me to directly execute javascript code in the html file as well. So, retrieving values that were dynamically generated became really easier.

For the design side of things, I initially created multiple css files according to their respective html pages. For example, habits.css for habits.html, categories.css for categories.html. But upon realization that most pages rendered similar styling to keep the visuals consistent, I trimmed it down to just base.css for overall styling across pages, style.css for the landing-page css and auth.css for signup and login pages. Besides, I used the external library "Bootstrap" to fast-forward the designing phase as it contains premade components like buttons, navbars, cards using flex display style etc.

Contribution of ID : 23201430, Name : Sumaya Afroze Chaha

Using HTML, CSS, JavaScript along with BootStraps and EJS, I have implemented several key features of the platform. One of the primary features is the budget setup, where users can define a fixed monthly budget based on their financial goals. This allows them to manage

their spending more effectively. In addition, I developed an expense tracking system that enables users to record and categorize their daily expenses. The system dynamically updates and provides a clear overview of their spending patterns, helping users stay within their budget. Another feature I worked on is the leaderboard, it ranks users based on their streak points. Overall, these features work together to provide users with a more interactive and engaging experience, helping them manage both their habits and finances more efficiently.

Contribution of ID : 24101526, Name : Monaz Zerin Monir Bismah

Using HTML, CSS, JavaScript, EJS, and Bootstrap, I developed several useful features for my HabitCents project, such as a dashboard to show user progress, savings goals to help users plan their finances, and an over-expenditure tracker to monitor spending habits. The dashboard provides a quick overview of key information like completed habits and remaining budget. The savings goals feature allows users to set targets and track their progress visually, which helps keep them motivated. The over-expenditure feature makes it easy to identify when spending goes beyond the set budget. Overall, combining these tools helped me build a clean, interactive, and user-friendly application.

Backend Development

Contribution of ID : 24141020, Name : Mohammed Sabbir Ahmed

For backend development, I went with Node.js and it's frameworks/libraries like

- Express - to build the server side logic, handling methods, routing, APIs etc.
- Nodemon - Also a package of node used to automatically restart the server when any change is detected.
- Body-Parser - I intended on using body-parser but upon further research, I figured out that express comes with it's own parser and body-parser is now deprecated. So, although in the development phase I used body-parser at times, I eventually replaced it with a better alternative (express's builtin parser). So, this is redundant.
- EJS - ejs for templating and running javascript code in the html files
- Express-session - To track if the user is logged in or not.
- Mysql2 - to interact with the database

```
app.get("/habits", requireLogin, renderProtectedPage("habits")); // Habit tracking page
app.get("/expenses", requireLogin, renderProtectedPage("expenses")); // Expense tracking page
app.get("/leaderboard", requireLogin, renderProtectedPage("leaderboard")); // User rankings page
app.get("/savings", requireLogin, renderProtectedPage("savings")); // Savings goals page
```

Here app is a const variable which is basically initializing express, and we are routing the get requests to the respective pages. requireLogin() is a function that was created to check if the user is logged in or not since these pages can't be access without user authentication.

```
app.post("/add-habit", requireLogin, (req, res) => {
  // Add a new habit for the logged-in user
  const { name, difficulty, category } = req.body; // Habit data from form/AJAX
  const user_id = req.session.user.user_id; // Get user ID from session
  const habit_id = Date.now(); // Use timestamp as unique habit ID

  const difficulty_score = Number(difficulty);
  if (difficulty_score < 1 || difficulty_score > 10) {
    // Validate difficulty is between 1-10
    return res.redirect("/habits");
  }

  // Insert new habit into database
  db.query(
    "INSERT INTO habits (habit_id, user_id, habit_name, difficulty_score, category) VALUES (?, ?, ?, ?, ?)",
    [habit_id, user_id, name, difficulty_score, category],
    (err) => {
      if (err) {
        console.error(err);
        return res.redirect("/habits");
      }
      res.redirect("/habits"); // Redirect back to habits page
    },
  );
});
```

Now, if we look at this code, this is one of the routes that I had to implement in order to get the "Add Habit" function working. So, this registers a route that listens for POST requests and verifies if the user is logged in using the requireLogin function discussed earlier. After those are verified, a callback function is executed. This callback function is built into express and has a request, response cycle. When the user clicks the form, a POST request is generated and this callback function is executed. Here, req contains the information that was sent with the POST request, in this case, values like name, difficulty, category. Then we retrieve the user_id from the session and generate a habit_id using javascripts builtin Date function which will give us the current time in milliseconds. This is basically used to create a unique identifier. Finally, as the output received from the user for difficulty scoring is a string, we have to convert it into a numerical value in order to do further calculation (measuring streak). Then , the function just checks whether the value is between 1-10 because that is a limit we set for the difficulty scoring. And if the user tries to set any value other than 1-10 , the user is redirected back to the page using the res.redirect function.

Next, the db.query is where the framework tries to connect and do modifications with the database. The expected values in the query has been kept '?' to deal with SQL injection attacks. Alternatively, the required values are passed in through an array. The rest of the code just deals with error handling.

Similarly, other routes have been created for different functionalities keeping almost identical structure. But, as the routes only manage the database queries , it is not sufficient to create the whole function using them. So, I also created more javascript code in the frontend as well to deal with events, actions etc. These sends requests to the server , the server then processes those requests using the code written in server.js and updates the db accordingly, after the database is updated, the updated data and response is then sent back to the frontend where the frontend JS makes necessary changes to display the changed values/output to the user. My contributions in the backend include-

- server.js
- habits.js
- habits.ejs

Contribution of ID : 23201430, Name : Sumaya Afroze Chaha

For Backend, I have used Node.js with Express for the server side and MySQL for the database management. For the budget setup, I created routes that allow users to set their monthly budget. This data is sent to the server and stored in the MySQL database, making sure each user has their own budget. The system then uses this value to track how much they can spend. In the expense tracking feature, users can add their daily expenses, which are saved in the database with description and amount. The backend continuously processes this data to calculate the total spending and keeps everything updated so users can clearly see where their money is going and how much is remaining. For the leaderboard, I implemented logic that ranks users based on their streak points. The backend collects and processes this data, then sends top 20 users' names according to the streak points, sorting in descending order and users can view their position. Overall, these features rely heavily on the backend to store user data, process it correctly, and send meaningful results to the frontend, making the platform both interactive and useful.

Contribution of ID : 24101526, Name : Monaz Zerin Monir Bismah

For the savings goals, budget, and over-expenditure features, the backend plays an important role in handling data and logic. Using JavaScript with Node.js and Express, I created routes that receive user inputs (like saving goals or setting a budget) through requests. This data is then stored in the database with proper user identification so each user has their own records. For savings goals, the backend stores the goal amount and tracks how much the user has saved, updating the progress whenever new data is added. For the budget feature, it saves the monthly budget and calculates how much money is left based on the user's expenses. In the case of over-expenditure, the backend compares the total expenses with the set budget and determines if the user has exceeded their limit. It then sends this processed information to the frontend, where it is displayed to the user in a clear and meaningful way.

Source Code Repository

Repository link

Conclusion

Lastly, our project successfully brings together all the features and works as intended. Throughout the development process, we explored and learned a wide range of technologies, including CSS, JavaScript, Node.js, and Express. This project gave us valuable hands-on experience in both frontend and backend development. We also applied important database concepts such as EER diagrams and Schema Mapping, which helped us design and manage our data in a structured and efficient way. Overall, it was a great learning experience that allowed us to turn our ideas into a functional and practical application.

References

1. Official Documentation: https://getbootstrap.com/docs/5.3
2. Icons Library: https://icons.getbootstrap.com
3. Official Examples: https://getbootstrap.com/docs/5.3/examples
4. Navbar Component: https://getbootstrap.com/docs/5.3/components/navbar
5. Cards Component: https://getbootstrap.com/docs/5.3/components/card
6. Forms Overview: https://getbootstrap.com/docs/5.3/forms/overview
7. Grid System: https://getbootstrap.com/docs/5.3/layout/grid
8. Buttons: https://getbootstrap.com/docs/5.3/components/buttons
9. Typography: https://getbootstrap.com/docs/5.3/content/typography
10. Color Utilities: https://getbootstrap.com/docs/5.3/utilities/colors
11. Flexbox Utilities: https://getbootstrap.com/docs/5.3/utilities/flex
12. MDN HTML Guide: https://developer.mozilla.org/en-US/docs/Web/HTML
13. W3Schools HTML: https://www.w3schools.com/html
14. MDN CSS Guide: https://developer.mozilla.org/en-US/docs/Web/CSS
15. W3Schools CSS: https://www.w3schools.com/css
16. CSS Flexbox Guide (MDN): https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout
17. Box Model Explained: https://css-tricks.com/the-css-box-model/

18. MDN JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
19. JavaScript.info: https://javascript.info
20. W3Schools JS: https://www.w3schools.com/js
21. Promises & Async/Await: https://javascript.info/async
22. DOM Manipulation: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
23. JS Array Methods: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
24. JS Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
25. Node.js Official: https://nodejs.org
26. Node.js Documentation: https://nodejs.org/en/docs
27. NPM Cheatsheet: https://devhints.io/npm
28. W3Schools Node.js: https://www.w3schools.com/nodejs
29. Express Official: https://expressjs.com
30. Express API Reference: https://expressjs.com/en/api.html
31. Routing Guide: https://expressjs.com/en/guide/routing.html
32. Middleware Guide: https://expressjs.com/en/guide/using-middleware.html
33. Express Error Handling: https://expressjs.com/en/guide/error-handling.html
34. Express GitHub: https://github.com/expressjs/express
35. Express Session: https://www.npmjs.com/package/express-session
36. EJS Official: https://ejs.co
37. EJS Cheatsheet: https://devhints.io/ejs
38. MariaDB Knowledge Base: https://mariadb.com/kb/en
39. MySQL Documentation: https://dev.mysql.com/doc
40. MySQL2 NPM Package: https://www.npmjs.com/package/mysql2
41. XAMPP Official: https://www.apachefriends.org
42. SQL Joins Guide: https://www.sql-join.com
43. W3Schools SQL: https://www.w3schools.com/sql
44. MySQL Data Types: https://www.w3schools.com/mysql/mysql_datatypes.asp
45. Nodemon (Auto-restart): https://nodemon.io